Doramagic v2 产品定义

基于 22 轮第一性原理灵魂拷问

2026-03-11

综合 22 轮灵魂拷问全部洞察 | 终稿

输入: discussion-log.md + doramagic-v2-redesign.md + v2-tech-gap-analysis.md

Tang x AI 产品专家 (Claude Opus)

目录

1. 一句话定义

2. 产品哲学（来自灵魂拷问的核心洞察）

2.1 知识魔法 vs 方法论魔法（Q8-Q9）

2.2 抄作业高手，非从零构建（Q3, Q17-Q18）

2.3 有立场的专家，非信息搬运工（Q17）

2.4 连接即知识——不可分割（Q15）

2.5 操作化 > 知识增量（Q14，Claude Code 类比）

2.6 好 vs 适合——千人千面（Q7-Q8）

2.7 越用越好用——自我学习（Q19）

2.8 30 -> 60, 80 -> 85 的价值公式（Q18）

3. 品牌定位：哆啦A梦（Q11）

3.1 品牌映射

3.2 触发方式

3.3 核心隐喻

4. 目标用户

4.1 主力用户：OpenClaw 终端用户

4.2 用户画像：老王

4.3 两类用户的价值差异

5. 核心能力

5.1 需求理解

5.2 多源灵魂提取

5.3 智能装配（Assembly, not Creation）

5.4 质量保证

5.5 回炉改造（Q9）

6. 输出形态

6.1 当前状态：Skill vs 专家配置

6.2 可能的混合方案

6.3 开袋即食，不需要用户配置

7. 技术架构原则

7.1 参考 Claude Code 工程实践（Q14）

7.2 MCP-first 资源层

7.3 Orchestrator + 自适应循环（非线性流水线）

7.4 自我学习与 MEMORY.md 反馈

7.5 AST-aware 代码理解

7.6 技术升级优先级

8. 关键需求清单

8.1 学习 Claude Code 工程模式（Q14）

8.2 自我学习能力 + 定期 review（Q19）

8.3 回炉改造功能（Q9）

8.4 多源提取（Q16）

8.5 用户意图识别（Q17-Q18）

8.6 透明展示能力边界（Q4）

8.7 OpenClaw 原生能力利用（Q5 技术差距分析）

9. 0-1 阶段核心风险（Q21-Q22）

最大风险：提取质量 + 装配质量

策略：两手都要抓，迭代验证

10. 开放问题

10.1 输出形态：Skill vs 专家配置 vs 混合体

10.2 护城河（待深入）

10.3 MVP 领域选择

10.4 Doramagic 道具 vs 直接问 AI 的竞争关系

10.5 "知识不是新闻"的时效性假设

11. 与 v1.0 对比

附录：22 轮讨论索引

1. 一句话定义

Doramagic 是 OpenClaw 生态中的哆啦A梦——理解用户的模糊需求，从开源项目、优秀 Skill、API 和社区经验中"抄最好的作业"，像拼乐高一样自动装配出开袋即食的个性化 AI 道具，让用户的 AI 变成真正懂行的领域专家。

关键词解构：

哆啦A梦：品牌人设，不教大雄做事，给他道具（Q11）

抄最好的作业：超级 Copy Cat，选择与适配而非从零创造（Q3, Q17-Q18）

知识魔法：注入真实能力，不是强加流程（Q8-Q9）

开袋即食：用户零配置即可使用（Q4）

个性化："好"和"适合"是两个层次，千人千面（Q7-Q8）

与 v1.0 定义的区别：

	v1.0	v2.0
定义	开源项目灵魂提取系统	OpenClaw 的哆啦A梦——个性化 AI 道具制造器
本质	引擎层描述（技术导向）	产品层描述（用户导向）

2. 产品哲学（来自灵魂拷问的核心洞察）

22 轮第一性原理审查沉淀出八条产品哲学，每一条都来自具体的讨论和真实的洞察。

2.1 知识魔法 vs 方法论魔法（Q8-Q9）

> superpowers 是"方法论魔法"——告诉你怎么做，强加流程。
> Doramagic 是"知识魔法"——给你能力去做，注入知识和资源。

这是 Doramagic 与所有现有 Skill 的根本区分。方法论 Skill 越严格越容易被删除（因为它约束用户）；知识 Skill 越丰富越被依赖（因为它赋能用户）。Tang 亲身经历了从"superpowers 好用"到"彻底删除"的全过程——因为无论什么事情都需要遵循它的规则，大部分工作根本不需要那么复杂的流程。

设计推论：Doramagic 生成的道具不能是固定 stage 流水线，必须是知识驱动的自适应行为——AI 自己判断什么时候走完整流程，什么时候直接干。

2.2 抄作业高手，非从零构建（Q3, Q17-Q18）

> Doramagic 是一个抄作业的高手，而非从 0 开始构建的高手。

核心能力是选择和适配，而非创造。面对矛盾来源时，Doramagic 的工作是选最好的作业来抄，而非发明新理论。这限定了清晰的能力边界：不发明、不创造——发现最好的现有方案并为用户量身适配。

设计推论：Doramagic 的成功取决于"可抄的好作业"的丰富程度，而非 AI 的创造力。

2.3 有立场的专家，非信息搬运工（Q17）

> 如果只是给用户买回来一堆零件，Doramagic 毫无价值。

大多数 AI 产品不敢做判断——列出 A/B/C 三个选项说"取决于你的需求"。这是推卸责任。用户要的不是零件，是成品。成品意味着有人替你做了判断。Doramagic 必须是一个有立场的专家，替用户做出选择。

设计推论：Doramagic 做出正确判断的前提是充分理解用户意图，因此苏格拉底对话不是可选项，是必选项。

2.4 连接即知识——不可分割（Q15）

> "你的理解肤浅了。连接过程本身就是一种知识。之所以需要大量提取灵魂，本质就是获取这种知识。"

"知道该用 akshare 而不是 tushare 来拿 A 股数据"——既是知识，也是连接能力。"知道东方财富 API 限频约 1 req/sec"——既是社区踩坑经验（知识），也是正确配置 API 连接的前提（连接）。二者不可分割，都融在项目的"灵魂"里。

设计推论：Soul Extractor
不需要分别提取"知识"和"连接"——它提取的是整体的、活的专业能力。灵魂的六个知识层（见第 5 节）缺一不可。

2.5 操作化 > 知识增量（Q14，Claude Code 类比）

> 如果大模型可以藐视一切，那 Claude Code 这样的工具就毫无价值。而事实上大模型越强，流程和产品化的价值更大。

这是 22 轮讨论中最精准的一击。LLM"知道"股票分析，但 Doramagic 的价值不在于补充 LLM 不知道的那 10% 知识，而在于将 LLM 已有的知识操作化——连接真实 API、注入特定数据源、创建工作流、添加领域特定工具链。正如 Claude（模型）知道怎么写代码，而 Claude Code（产品）加上了文件系统、Git、工具编排、持久上下文，将编程知识产品化为 10 倍有用的东西。

公式：Doramagic 的价值 = LLM 已有知识 x 操作化基础设施，而非 LLM 已有知识 + 知识增量。

2.6 好 vs 适合——千人千面（Q7-Q8）

> "好"和"适合"是两个完全不同的层次。

在没有 AI 的时代，用户只能忍受不适合的功能。AI 时代这一定会改变。"适合"的本质不是"个性化内容"，而是"自适应行为"——AI 能判断这件事需要走完整流程，还是直接干就行。superpowers 的失败不是因为 TDD 不好，而是因为它不知道什么时候不该用 TDD。

设计推论：Doramagic 不应该教用户开发 Skill，而是直接给他成品来解决问题。用户说"帮我选股"，他就得到一个能选股的东西。句号。

2.7 越用越好用——自我学习（Q19）

> 任何 AI 智能体都应该具有"越用越好用"的特性，使用过程就是双方提升的过程。

道具的进化路径：

1. 先让道具足够好，建立基本信任（30 -> 60 的价值已经成立）
2. 通过回炉改造，逐步提升道具质量（60 -> 70 -> 80）
3. 信任阈值在互动中自然校准

产品节奏：不在 Day 1 解决全部质量问题，而是通过演进式互动持续提升。

2.8 30 -> 60, 80 -> 85 的价值公式（Q18）

> 帮低级用户从 30 分到 60 分，帮高级用户从 80 分到 85 分。

不追求完美，追求 delta。大部分用户需要的不是 100 分完美方案，而是"比自己能做到的好很多"。低级用户低质量输入，高级用户高质量输入——Doramagic

在两端都提供价值，只是 delta 的性质不同。

3. 品牌定位：哆啦A梦（Q11）

3.1 品牌映射

Doramagic 的名字来自 Doraemon（哆啦A梦）。这不仅是品牌——这是产品哲学的完美映射。

哆啦A梦	Doramagic
大雄带着模糊的烦恼来找哆啦A梦	用户带着模糊的需求触发 Dora
哆啦A梦理解真实需求（不是大雄说的那个）	苏格拉底对话发现真实问题
从四次元口袋里拿出最合适的道具	从多源提取知识+资源，锻造专家配置
道具交给大雄，大雄自己用	配置注入 OpenClaw，AI 自主运行
有时道具被滥用，哆啦A梦警告	透明展示能力边界 + 可选升级提示
大雄遇到新问题再来找哆啦A梦	回炉改造

核心叙事：哆啦A梦从不教大雄怎么做事（不是方法论魔法），而是给他能力去解决问题（知识魔法）。

3.2 触发方式

/dora 或 "Dora，你帮我XXX"

提供多语言版本

人设和说话风格：模拟哆啦A梦和大雄的对话——亲切、直接、偶尔吐槽

3.3 核心隐喻

四次元口袋 = Doramagic 的多源知识/资源库

每次拿出的道具 = 为用户锻造的个性化专家配置

每次只掏一个道具 = 聚焦解决一个具体问题，而非一揽子方案

4. 目标用户

4.1 主力用户：OpenClaw 终端用户

非技术用户优先。 Doramagic 的用户永远不应该接触"Skill 开发"的概念。

4.2 用户画像：老王

> 老王，45 岁，中国 A 股散户，没有任何产品或代码能力。他总是不能让 OpenClaw 按照他的要求完成股票研究、股票跟踪、选股等任务。现有的通用 Skill 无法满足他个性化的投资分析需求。

老王代表的核心断层：现有 Skill 市场是开发者预制的通用产品，用户只能下载使用，无法做到完全个性化和量身定制。

4.3 两类用户的价值差异

用户类型	输入质量	Doramagic 价值	典型场景
低级用户（	低质量、模	30 -> 60 分（巨大 del...	"帮我炒股"
高级用户	高质量、精	80 -> 85 分（精细 del...	"帮我搭一个基于 PE/PB 多因子的 A 股价值投资选股系统，数据用 aks...

两端都有价值，但最大的价值场景（30 -> 60）恰好也是最需要小心的场景——这些用户没有能力评估输出质量。解决方式：先建立信任，再通过使用逐步校准（见 2.7 节）。

5. 核心能力

5.1 需求理解

苏格拉底对话 + 意图识别。

Doramagic 做出正确判断的前提是充分理解用户意图。3-5 轮对话，每轮 1-2 个问题：

轮次	目的	示例
R1	问题本质	"你想让 AI 帮你做什么？最痛的一件事是什么？"
R2	场景与频率	"这件事你多久做一次？花多长时间？现在怎么做的？"
R3	成功标准	"做到什么程度你会觉得'这真好用'？"
R4	偏好与约束	"你更想要简单快速的还是功能全面的？"
R5	确认	"我理解你需要 X，核心是 Y，对吗？"

硬门控：需求画像必须包含 core_problem + trigger + expected_output，否则继续对话。

5.2 多源灵魂提取

这是 v2 相比 v1 最大的扩展——从 GitHub-only 扩展到多源。

灵魂的六个知识层

v1.0 只覆盖了前三层，v2 必须覆盖全部六层：

知识层	含义	v1.0 覆盖度	v2 目标
领域知识（该知道什么）	概念、理论、设计哲学	部分	完整
操作知识（怎么做）	API 调用、工具配置、集成方法	弱	完整
经验知识（什么有坑）	社区踩坑、已知陷阱、边界情况	部分	完整
决策知识（何时用什么）	选型判断、权衡规则、最佳实践	弱	完整
生态知识（什么连什么）	工具链组合、依赖关系、兼容性	极弱	完整
时效知识（什么已过时）	版本变更、API 废弃、新替代方案	未覆盖	基础覆盖

四梯队来源模型

灵魂不仅住在 GitHub 里。老王的灵魂来自整个领域生态。

梯队	来源	价值定位
T1 必...	MCP Server 注册表（最关键新来源）、GitHub 仓库（扩展提取维度）、OpenClaw Skill...	核心能力来源
T2 高...	API 文档（OpenAPI）、Stack Overflow、Awesome-lists	操作知识和生态知识...
T3 补...	技术博客、领域垂直社区（雪球/集思录等）、包管理器元数据	经验知识和时效知识
T4 未来	会议演讲、论坛讨论、行业法规	长尾覆盖

关键洞察：MCP Server 注册表是 v2 最重要的新来源——一个 MCP Server = 一份结构化的"连接+操作知识"，可直接被 AI 调用，不需要再手工提取。

5.3 智能装配（Assembly, not Creation）

核心是装配，不是创造。

拼装逻辑四步：

1. 蓝图设计：从需求画像提取 trigger、阶段骨架、输出格式；从模式卡片提取参考架构
2. 资源绑定：选择最优资源（优先：免费 + 低集成复杂度 + 高可靠性）；MCP-first 接入
3. 规则注入：将关键决策规则、社区踩坑经验、已知陷阱写入道具
4. 生成道具：产出可运行的配置包

"开袋即食"原则：

免费、无需认证的资源：默认启用

需要 API Key 的资源：标记为"可选升级"，不影响基础功能

需要用户数据的功能：提示"提供 X 可解锁 Y 能力"

5.4 质量保证

> 没有 AI，50% 概率选错；有了 AI 应该变成 30% 概率选错。Doramagic 没法避免选错，但能极大减少选错概率。——Q17

质量保证机制：

硬门控：每个阶段设质量关卡（需求画像完整性、源泉最低数量、卡片最低数量、冒烟测试通过）

社区验证：注重社区踩坑和最佳实践——社区验证的经验就是降低错误率的核心机制

证据级别：每条知识标注来源和可信度

冒烟测试：生成的道具必须通过基本功能验证才能交付

5.5 回炉改造（Q9）

经 Doramagic 制造的道具可以重新进入 Doramagic 进行改造，以用户使用数据为依据来完善。

进化循环：

```
Doramagic 生成道具 v1.0 -> 用户使用 -> 积累使用数据/反馈
-> 回炉 Doramagic -> 生成道具 v1.1（更适合用户）-> ...
```

不是"自动优化"，而是"回炉改造"——用户主动触发，Doramagic 基于数据执行。

6. 输出形态

6.1 当前状态：Skill vs 专家配置

这是一个尚未完全关闭的设计讨论：

Skill 形态：Tang 原始构想，创造一个用户定制的 Skill（Q12）

专家配置形态：SOUL.md + AGENTS.md + MEMORY.md + MCP 配置，不是独立 Skill，而是"让 OpenClaw 自身变成专家"（Q10-Q11）

6.2 可能的混合方案

> Skill 的外壳 + 知识驱动的内核：不是固定 stage 流水线，而是 SOUL.md 级别的知识注入。

这意味着：

外壳是 OpenClaw 原生 Skill 格式（因为这是平台的原生单元）

内核不是写死的阶段流程，而是注入的领域知识 + 资源连接 + 决策规则

AI 根据知识自主决定行为（自适应），而非按预设阶段硬跑

6.3 开袋即食，不需要用户配置

无论最终形态如何，核心原则不变：

交付即可用，零配置门槛

可选升级清晰提示，但不影响基础功能

能力边界透明展示

7. 技术架构原则

7.1 参考 Claude Code 工程实践（Q14）

> 产品 = 模型 + 文件系统 + Git + 工具编排 + 项目上下文

Doramagic 应参考 Claude Code 的工程模式：不是简单包装 LLM，而是为 LLM 提供操作化基础设施——让 AI 能真正"做事"而不只是"说话"。

7.2 MCP-first 资源层

资源通过 MCP 协议接入，而非内嵌代码调用。

```
原方案: Skill 内嵌 import akshare; ak.stock_zh_a_spot_em()
升级方案: Skill 调用 mcp://stock-data/realtime?market=A
      -> 背后对接 akshare MCP Server（可独立更新/替换）
```

好处：资源可独立更新、可跨道具复用、符合 OpenClaw 原生集成模式。

7.3 Orchestrator + 自适应循环（非线性流水线）

2026 的 Agent 架构不是 5 阶段串行流水线，而是 Orchestrator 自适应循环：

- 在拼装阶段发现知识不够，自动回到提取阶段补充
- 在交付阶段发现冒烟测试失败，自动回到拼装阶段修复
- 子代理通过共享工作记忆通信，而非文件传递

```
      Orchestrator Agent (总控)
      动态调度，非固定顺序
  ┌──────────┬──────────┬──────────┐
  v     v     v     v
需求   发现   提取   拼装
Agent  Agent  Agent(s)  Agent
  |     |     |     |
  └──────────┴──────────┴──────────┘
      | 共享工作记忆
      v
  Delivery Agent
```

7.4 自我学习与 MEMORY.md 反馈

- 道具运行时将用户反馈写入 MEMORY.md
- 下次运行读取记忆，动态调整行为
- 定期触发"道具体检"——检查资源是否仍然可用

越用越好用，使用过程就是双方提升的过程

7.5 AST-aware 代码理解

代码是结构化知识图，不是一坨文本。

```
原方案: git clone -> repomix compress -> 一个大 XML -> 喂 LLM
升级方案: git clone -> AST 分析（函数/类/模块结构）
          -> 代码嵌入（语义索引）
          -> Agent 按需查询（"找到所有 API 端点定义"）
          -> 只把相关代码片段喂 LLM
```

解决大仓库 token 限制问题，提升提取精准度。

7.6 技术升级优先级

优先级	升级项	理由
P0	MCP-first 资源层	"抄作业引擎"的核心——抄来的能力可直接运行
P1	Orchestrator 自适应循环	线性流水线无法自愈
P2	反馈闭环 + 自我优化	从"工具"变成"产品"的关键，决定留存
P3	AST 级代码理解	提升提取质量上限
P4	语义发现 + 能力图谱	长期竞争壁垒，冷启动可用 LLM 知识替代

8. 关键需求清单

8.1 学习 Claude Code 工程模式（Q14）

Doramagic 开发时应深入参考 Claude Code 的工程实践：

模型能力与系统工具的无缝集成

权限模型、工具编排模式、上下文管理策略

"知识操作化"作为产品设计核心原则

8.2 自我学习能力 + 定期 review（Q19）

道具具备自我学习能力——越用越好用

定期 review 机制：道具根据使用数据自检，发现短板

MEMORY.md 反馈循环：使用中的经验沉淀为持久记忆

8.3 回炉改造功能（Q9）

用户可将使用中的道具带回 Doramagic 进行改进

以用户使用数据为依据，而非重新从零提取

形成 v1.0 -> v1.1 -> v1.2 的渐进进化

8.4 多源提取（Q16）

从 GitHub-only 扩展到四梯队多源模型

MCP Server 注册表作为最关键新来源

覆盖灵魂的全部六个知识层

8.5 用户意图识别（Q17-Q18）

苏格拉底对话是必选项，不是可选项

低级用户需要更多引导（30 -> 60 场景）

高级用户需要精准理解（80 -> 85 场景）

环境感知：读取用户 OpenClaw 配置、已安装 Skill、可用模型

8.6 透明展示能力边界（Q4）

四个呈现要求：

1. 足够专业：充分调用 OpenClaw 上的资源
2. 足够智能：尽可能少让用户反复交互，多用子代理干活
3. 足够适用：开袋即食 + 可选增强提示

4. 足够透明：让用户看到生产过程的关键进展，而非等 30 分钟得到一个黑盒结果

8.7 OpenClaw 原生能力利用（Q5 技术差距分析）

OpenClaw 能力	应用方式
MEMORY.md	道具运行记忆 + 用户偏好持久化 + 反馈闭环
AGENTS.md	定义子代理角色（提取代理、拼装代理等）
内置工具	源泉发现阶段直接调用 web search 等
MCP Server	资源层通过 MCP 接入
Skill 间调用	生成的道具可调用其他已安装 Skill
多模型调度	复杂任务用 Opus，简单任务用 Haiku/Flash

9. 0-1 阶段核心风险（Q21-Q22）

最大风险：提取质量 + 装配质量

> "灵魂提取的质量足以锻造可用的道具"——这是最危险的假设。

这个风险需要拆解为两个子风险：

风险	含义	验证方式
提取风险	从多源中提取的知识质量不够（提不出好零件）	在一个有充足好作业的领域进行端到端提取
装配风险	零件够好，但拼不成能用的成品（零件不兼容、缺关键部分、组装逻辑有问题	将提取结果实际拼装成可运行的道具

v1.0 已经证明"单源提取"是可行的（42% -> 96%）。但 v2 的挑战是多源提取 + 智能装配——这是 v1.0 从未验证过的。

策略：两手都要抓，迭代验证

Tang

拒绝做二选一的优先级排序——提取和装配是串联关系，不是并联。任何一环断了，产品就不成立。与其赌哪个先，不如两个同步推进、迭代验证。

10. 开放问题

10.1 输出形态：Skill vs 专家配置 vs 混合体

Skill 外壳 + 知识驱动内核是目前最可能的方向，但尚未做最终决策。需要在实际构建中验证哪种形态最自然、最符合 OpenClaw 平台约束。（Q10, Q12）

10.2 护城河（待深入）

Tang 坦承暂时思考深度不够。一个直觉方向：Doramagic 成为"AI Skill 积木仓库"——AI 获取、AI 贡献、AI 消费的原料库，类似 npm 之于 JavaScript。如果仓库足够大、质量足够高，就有网络效应。但"平台为什么不自己做"这个问题尚未回答。（Q6）

10.3 MVP 领域选择

0-1 阶段应选择一个"有充足好作业可抄"的领域进行验证。候选领域需满足：

- 开源生态丰富（有足够的好作业）
- 用户需求明确（痛点可感知）
- 非技术用户也能评价效果（可验证）

10.4 Doramagic 道具 vs 直接问 AI 的竞争关系

2026 年的 LLM 已经非常强大。"你是一个资深股票分析师..."这样的三行 prompt 也能角色扮演。

核心区分（Q13-Q14）：三行 prompt 给的是一个"听起来很懂的分析师"（演），Doramagic 给的是一个真正知道该调哪个 API 拿实时数据、从社区踩坑中学到了实战经验、知道哪些数据源靠谱的专家（真会）。一个是角色扮演魔法，一个是知识魔法。

但要持续保持这个差距，需要 Doramagic 的操作化能力始终领先于 LLM 原生能力的进化。

10.5 "知识不是新闻"的时效性假设

Tang 判断："华尔街精英们不是每天都在迭代知识，这做不到。"（Q14）

这意味着 Doramagic 提取的知识不需要每天更新（降低维护成本），知识保鲜期以年计。但也意味着"这种知识会因为 AI 的强大而发生巨大的变化"——随着 AI 进化，连接方式/工具链/最佳实践都在变，Doramagic 需要持续从最新源中提取。

11. 与 v1.0 对比

维度	v1.0	v2.0
一句话定	开源项目灵魂提取系统	OpenClaw 的哆啦A梦——个性化 AI 道具制造器
产品哲学	知识提取工具	知识魔法 + 抄作业高手 + 有立场的专家
品牌	无明确品牌人设	哆啦A梦，/dora 触发
入口	GitHub URL（用户主动提供）	用户的一句话需求（AI 主动发现）
出口	CLAUDE.md / 知识文档（给人看）	可运行的 AI 道具（给 AI 用）
目标用户	开发者 / CTO	OpenClaw 终端用户（非技术用户优先）
提取来源	单源（GitHub 仓库）	多源四梯队（MCP + GitHub + Skill 市场 + 社区 + API 文档）
知识覆盖	3/6 知识层（领域、部分经验）	6/6 知识层（含操作、决策、生态、时效）
核心能力	知识提取	需求理解 + 多源提取 + 智能装配 + 质量保证 + 回炉改造
缺失维度	资源（API/数据源/工具链）完全缺失	Resource Card / MCP Server 补齐
技术架构	线性 8+2 stage 流水线	Orchestrator 自适应循环 + 子代理团队
代码理解	Repomix 文本压缩	AST 分析 + 按需加载
资源集成	无	MCP-first，可执行接口
生命周期	一次生成，永不进化	生成 + 反馈 + 回炉改造 + 自我优化
交付体验	等 40 分钟得到文档	实时进度 + 开袋即食
用户角色	主动型（用户要知道分析哪个项目）	被动型（AI 帮用户找到该抄谁的作业）
价值公式	42% -> 96%（AI 回答准确率）	30 -> 60 / 80 -> 85（用户能力 delta）

附录：22 轮讨论索引

轮次	主题	关键产出
Q1	用户到底在为什么付费？	从"灵魂提取工具"转向"OpenClaw 个性化 agent 制造器"
Q2	"个性化 agent"需求验证	老王（股民）案例引入
Q3	能力与需求的逻辑断裂	"超级 Copy Cat"概念诞生
Q4	谁来决定抄什么、怎么拼？	四个呈现要求：专业、智能、适用、透明
Q5	技术先进性审查	六个关键差距识别，技术落后 1-2 代
Q6	护城河在哪里？	"AI Skill 积木仓库"直觉，待深入
Q7	需求验证方法论	"好 vs 适合"洞察，创始人体验即验证
Q8	"适合"的本质	自适应行为 > 个性化内容
Q9	会不会犯 superpowers 的错？	知识魔法 vs 方法论魔法区分；回炉改造
Q10	Skill 到底应该长什么样？	Doramagic 边界问题浮现
Q11	Doramagic 的能力边界	哆啦A梦品牌定位确立
Q12	下一步到底做什么？	Skill vs 专家配置争议；继续灵魂拷问
Q13	道具 vs 直接问 AI	角色扮演魔法 vs 知识魔法
Q14	知识增量的价值	Claude Code 类比——操作化 > 知识增量
Q15	知识工具 vs 连接工具	"连接即知识"——不可分割
Q16	v1.0 提取范畴够宽吗？	六个知识层 + 四梯队来源模型
Q17	多源提取的一致性	"有立场的专家"决策；50% -> 30% 错误率
Q18	用户意图识别深度	"抄作业高手"强调；30->60, 80->85 价值公式
Q19	60 分道具的置信度	自我学习能力；先信任再校准
Q20	没有好作业可抄怎么办	0-1 阶段不讨论天花板
Q21	0-1 最大风险	提取质量 + 装配质量是最危险假设
Q22	风险拆解	两手都要抓，迭代验证